

分支任务1：gdb 调试页表查询过程

一、实验目的与背景

1.1 实验目的

- 1.深入理解虚拟地址到物理地址的转换流程，包括 TLB 查询、多级页表遍历、TLB 更新等核心环节。
- 2.掌握“双重 GDB 调试”技术，即同时调试 QEMU 模拟器源码与运行在其中的 ucore 内核，建立硬件模拟与软件执行的关联认知。
- 3.培养利用大模型解决复杂技术问题的能力，学习在大模型辅助下梳理调试流程、定位关键代码、解决调试故障。

1.2 实验背景

在虚拟内存访问模式下，CPU产生的虚拟地址需要经过MMU转换为物理地址。在真实的硬件环境中，这一过程由硬件自动完成：

- 首先查询TLB（转址旁路缓存）
- 若TLB未命中，则查询多级页表
- 将转换结果存入TLB并完成访存

在实验中，我们通过QEMU软件模拟硬件行为，这使我们有机会通过调试工具深入观察这一过程。

二、实验环境准备

2.1 编译带调试信息的QEMU

首先需要重新编译一个带调试信息的QEMU。之前安装QEMU的时候用的是正常的配置，虽然能用但是没有调试符号，GDB没法在里面设置断点。

```
cd ~/qemu-4.1.1
make distclean
./configure --target-list=riscv32-softmmu,riscv64-softmmu --enable-debug
make -j$(nproc)
```

这里有个重要的点：**编译完成后不要执行 `sudo make install`！**我们只需要一个带调试信息的可执行文件用来调试，不是要替换系统里原有的QEMU。编译完成后，在 `riscv64-softmmu` 目录下可以找到 `qemu-system-riscv64` 这个文件。

2.2 修改Makefile

接下来需要修改ucore项目的Makefile，让它使用我们新编译的这个调试版QEMU：

```
QEMU := /home/1ht/qemu-4.1.1/riscv64-softmmu/qemu-system-riscv64
```

这样在执行 `make debug` 的时候，就会用带调试信息的QEMU了。

三、调试流程设计与实施

3.1 整体调试架构设计

本次实验采用双重GDB调试架构：

1. GDB会话1：使用系统原生GDB，附加到QEMU进程，用于调试QEMU源码
2. GDB会话2：使用riscv64-unknown-elf-gdb，通过QEMU的GDB stub接口连接，用于调试uCore内核
3. 三个终端协同工作，形成完整的调试链路

3.2 终端启动

步骤1：启动QEMU（终端1）

在uCore项目目录下执行：

```
make debug
```

该命令启动调试版QEMU，并暂停在初始状态等待GDB连接。

步骤2：附加调试QEMU进程（终端2）

首先查找QEMU进程的PID：

```
pgrep -f qemu-system-riscv64
```

然后启动GDB并附加到该进程：

```
sudo gdb
```

在GDB会话中执行：

```
(gdb) attach <PID>
(gdb) handle SIGPIPE nostop noprint # 处理SIGPIPE信号，避免不必要的中断
(gdb) c
Continuing...
```

步骤3：调试uCore内核（终端3）

在终端2执行 `continue` 启动QEMU执行后，在终端3中：

```
make gdb
```

这会启动riscv64-unknown-elf-gdb并连接到QEMU的GDB stub。

启动终端3之后，我们可以继续在终端3中设置断点，进行操作：

```
(gdb) break kern_init          # 设置断点
(gdb) continue                 # 开始执行
```

开始执行后，我们可以迅速找到对应断点，并停止，我们可以使用下面的指令来查看

```
(gdb) info registers pc          # 查看PC寄存器
(gdb) x/10i $pc                 # 查看接下来10条指令
```

结果如下图所示：

```
(gdb) b kern_init
Breakpoint 1 at 0xffffffffc02000d8: file kern/init/init.c, line 30.
(gdb) c
Continuing.

Breakpoint 1, kern_init () at kern/init/init.c:30
30      memset(edata, 0, end - edata);
(gdb) info registers pc
pc          0xffffffffc02000d8      0xffffffffc02000d8 <kern_init>
(gdb) x/10i $pc
=> 0xffffffffc02000d8 <kern_init>:      auipc    a0,0x6
    0xffffffffc02000dc <kern_init+4>:      addi     a0,a0,-192
    0xffffffffc02000e0 <kern_init+8>:      auipc    a2,0x6
    0xffffffffc02000e4 <kern_init+12>:     addi     a2,a2,-104
    0xffffffffc02000e8 <kern_init+16>:     addi     sp,sp,-16
    0xffffffffc02000ea <kern_init+18>:     sub      a2,a2,a0
    0xffffffffc02000ec <kern_init+20>:     li       a1,0
    0xffffffffc02000ee <kern_init+22>:     sd       ra,8(sp)
    0xffffffffc02000f0 <kern_init+24>:     jal      ra,0xffffffffc020171c <memset>
    0xffffffffc02000f4 <kern_init+28>:
    jal      ra,0xffffffffc0200220 <dtb_init>
```

从上图所示的调试信息中可以看出，程序已成功进入内核的虚拟地址空间地址为 `0xffffffffc02000d8`，并且通过反汇编展示了该位置开始的10条RISC-V汇编指令序列。

通过分析这段指令流，可以发现 `sd ra, 8(sp)` 是其中唯一一条执行内存存取操作的指令，它的作用是将返回地址寄存器 `ra` 的值保存到栈中偏移8字节的位置。由于我们需要进行访存指令级别的调试，可以通过单步执行的方式逐条执行指令，直至运行到该访存指令，以便观察或验证内存写入的具体行为。

```
# 单步执行到某个访存指令
(gdb) si                                # 单步执行一条指令
```

3.3 调试会话协同策略

通过询问大模型，我理解了三终端协同的工作流程：

1. 终端3先控制uCore执行到特定访存指令
2. 访存发生时，终端2的GDB在设置的关键函数断点处中断
3. 在终端2中单步执行QEMU源码，观察地址转换过程
4. 继续执行后，控制权回到终端3

四、观察地址翻译流程

4.1 找到访存指令

在终端3（uCore调试终端）中，使用以下命令查看后续的10条指令：

```
(gdb) x/10i $pc
```

```
(gdb) x/10i $pc
=> 0xfffffffffc02000e0 <kern_init+8>:  auipc  a2,0x5
    0xfffffffffc02000e4 <kern_init+12>:  addi   a2,a2,-104
    0xfffffffffc02000e8 <kern_init+16>:  addi   sp,sp,-16
    0xfffffffffc02000ea <kern_init+18>:  sub    a2,a2,a0
    0xfffffffffc02000ec <kern_init+20>:  li     a1,0
    0xfffffffffc02000ee <kern_init+22>:  sd     ra,8(sp)
    0xfffffffffc02000f0 <kern_init+24>:  jal    ra,0xfffffffffc0201650 <memset>
    0xfffffffffc02000f4 <kern_init+28>:
```

输出分析:

```
=> 0xfffffffffc02000e0 <kern_init+8>:  auipc  a2,0x5
    0xfffffffffc02000e4 <kern_init+12>:  addi   a2,a2,-104
    0xfffffffffc02000e8 <kern_init+16>:  addi   sp,sp,-16
    0xfffffffffc02000ea <kern_init+18>:  sub    a2,a2,a0
    0xfffffffffc02000ec <kern_init+20>:  li     a1,0
    0xfffffffffc02000ee <kern_init+22>:  sd     ra,8(sp)    ← 目标访存指令
    0xfffffffffc02000f0 <kern_init+24>:  jal    ra,0xfffffffffc0201650 <memset>
    0xfffffffffc02000f4 <kern_init+28>:  ...
```

我们选择地址 `0xfffffffffc02000ee` 处的 `sd ra,8(sp)` 指令作为观察目标。这是一条**存储指令** (Store Doubleword) , 将返回地址寄存器 `ra` 的值存储到栈指针 `sp+8` 指向的内存位置。

设置断点并执行到该位置:

```
0xfffffffffc02000f0 <kern_init+24>:  jal    ra,0xfffffffffc020171c <memse
0xfffffffffc02000f4 <kern_init+28>:
    jal ra,0xfffffffffc0200220 <dtb_init>
(gdb) b *0xfffffffffc02000ee
Breakpoint 2 at 0xfffffffffc02000ee: file kern/init/init.c, line 28.
(gdb) c
Continuing.

Breakpoint 2, 0xfffffffffc02000ee in kern_init () at kern/init/init.c:28
28      int kern_init(void) {
(gdb) x/i $pc
=> 0xfffffffffc02000ee <kern_init+22>:  sd     ra,8(sp)
```

```
(gdb) b *0xfffffffffc02000ee
Breakpoint 2 at 0xfffffffffc02000ee: file kern/init/init.c, line 28.
(gdb) c
Continuing.
Breakpoint 2, 0xfffffffffc02000ee in kern_init () at kern/init/init.c:28
(gdb) x/i $pc
=> 0xfffffffffc02000ee <kern_init+22>:  sd     ra,8(sp)
```

确认程序已停在目标访存指令处。

4.2 在QEMU中观察翻译过程

4.2.1 触发QEMU断点

在终端2（QEMU调试终端）中设置断点：

```
(gdb) break get_physical_address
(gdb) continue
```

回到终端3执行单步，终端2成功触发断点：

```
(gdb) c
Continuing.

Thread 1 "qemu-system-ris" hit Breakpoint 1, get_physical_address (
    env=0x57e57c6e0060, physical=0x7fff800f69f8, prot=0x7fff800f69f0,
    addr=18446744072637907160, access_type=0, mmu_idx=1)
    at /home/www/qemu-4.1.1/target/riscv/cpu_helper.c:158
158      {
(gdb) █
```

4.2.2 函数参数分析

`get_physical_address` 是QEMU中RISC-V架构的核心地址翻译函数，各参数含义：

参数	值	含义
env	0x57e57c6e0060	RISC-V CPU环境结构体指针，保存CPU状态和寄存器
physical	0x7fff800f69f8	输出参数，用于存储翻译后的物理地址
prot	0x7fff800f69f0	输出参数，存储内存保护属性（RWX权限）
addr	0xffffffffc02000d8	待翻译的64位虚拟地址
access_type	0	访问类型（0=读取/取指）
mmu_idx	1	MMU模式索引（1=S-mode内核态）

4.2.3 调用栈分析

我们可以利用下述的指令查看整体的一个流程：

```
(gdb) bt
```

```
(gdb) bt
#0  get_physical_address (env=0x5d367569d090, physical=0x7ffcb2262838,
    prot=0x7ffcb2262830, addr=18446744072637906944, access_type=0, mmu_idx=1)
    at /home/youth/qemu-4.1.1/target/riscv/cpu_helper.c:158
#1  0x00005d3664bb6c71 in riscv_cpu_get_phys_page_debug (cs=0x5d3675694680,
    addr=18446744072637906944)
    at /home/youth/qemu-4.1.1/target/riscv/cpu_helper.c:389
#2  0x00005d3664a7ad68 in cpu_get_phys_page_attrs_debug (cpu=0x5d3675694680,
    addr=18446744072637906944, attrs=0x7ffcb22628d8)
    at /home/youth/qemu-4.1.1/include/qom/cpu.h:607
#3  0x00005d3664a86a2f in cpu_memory_rw_debug (cpu=0x5d3675694680,
    addr=18446744072637907136, buf=0x7ffcb2262a04 "", len=64, is_write=0)
    at /home/youth/qemu-4.1.1/exec.c:3976
#4  0x00005d3664ad842b in target_memory_rw_debug (cpu=0x5d3675694680,
    addr=18446744072637907136, buf=0x7ffcb2262a04 "", len=64, is_write=false)
    at /home/youth/qemu-4.1.1/gdbstub.c:84
#5  0x00005d3664adb2b6 in handle_read_mem (gdb_ctx=0x7ffcb22629f0,
    user_ctx=0x0) at /home/youth/qemu-4.1.1/gdbstub.c:1771
#6  0x00005d3664ada837 in process_string_cmd (s=0x5d367593df50, user_ctx=0x0,
    data=0x5d367593df6c "ffffffffc02000c0,40",
    cmds=0x5d366540b8a0 <read_mem_cmd_desc>, num_cmds=1)
    at /home/youth/qemu-4.1.1/gdbstub.c:1476
#7  0x00005d3664ada8b8 in run_cmd_parser (s=0x5d367593df50,
    data=0x5d367593df6c "ffffffffc02000c0,40",
```

调用链解读：

GDB调试命令（读取内存）

```

↓
handle_read_mem()          ← GDB stub处理内存读取请求
↓
target_memory_rw_debug()   ← 目标机器内存读写调试接口
↓
cpu_memory_rw_debug()      ← CPU内存读写调试
↓
cpu_get_phys_page_attrs_debug() ← 获取物理页属性
↓
riscv_cpu_get_phys_page_debug() ← RISC-V特定的页表查询
↓
get_physical_address()     ← 核心：SV39页表遍历翻译

```

4.3 页表遍历详细过程

步骤1：读取关键寄存器

在函数入口处，用 GDB 打印这些参数：

```
(gdb) p/x addr          # 要翻译的虚拟地址
(gdb) p/x env->satp      # satp寄存器的值
(gdb) p access_type     # 访问类型：读/写/取指
(gdb) p/x addr
$1 = 0xffffffffc02000d8      # 要翻译的虚拟地址（内核高地址）

(gdb) p/x env->satp
$2 = 0x8000000000080204      # satp寄存器的值

(gdb) p access_type
$3 = 0                      # 读取操作
```

```
(gdb) p/x addr
$1 = 0xfffffffffc02000d8
(gdb) p/x env->satp
$2 = 0x80000000000080204
(gdb) p access_type
$3 = 0
```

SATP寄存器解析：

- `0x80000000000080204` 的二进制高4位为 `1000`，即MODE=8，表示**SV39分页模式**
- 低44位 `0x80204` 是根页表的物理页号（PPN）

步骤2：计算根页表基地址

```
base = get_field(env->satp, SATP_PPN) << PGSHIFT;
(gdb) p/x base
$5 = 0x80204000          # 根页表物理地址 = PPN << 12
```

计算验证：`0x80204 << 12 = 0x80204000` ✓

```
184         base = get_field(env->satp, SATP_PPN) << PGSHIFT;
(gdb) s
185         sum = get_field(env->mstatus, MSTATUS_SUM);
(gdb) p/x base
$5 = 0x80204000
(gdb)
```

步骤3：页表遍历循环

第一次迭代 (i=0, 查询VPN[2])：

```
(gdb) p i
$6 = 0          # 第一级页表（最高级，VPN[2]）

(gdb) p/x idx
$7 = 0x100      # VPN[2] = VA[38:30] = 0x100 (256)

(gdb) p/x pte_addr
$8 = 0x80204800 # 页表项地址 = base + idx*8

(gdb) p/x pte
$9 = 0x200000cf # 页表项内容
```

```
(gdb) p i
$6 = 0
(gdb) p /x idx
$7 = 0x57e56a7a7fa9
(gdb)
```

页表项(PTE)标志位解析：

```
(gdb) p/x (pte&1)
$2 = 0x1                                # v=1 (valid, 有效)

(gdb) p/x (pte&(1<<1))
$3 = 0x2                                # R=1 (Readable, 可读)

(gdb) p/x (pte&(1<<2))
$4 = 0x4                                # W=1 (Writable, 可写)

(gdb) p/x (pte&(1<<3))
$5 = 0x8                                # X=1 (Executable, 可执行)
```

```
(gdb) p/x (pte&1)
$2 = 0x1
(gdb) p/x (pte&(1<<1))
$3 = 0x2
(gdb) p/x (pte&(1<<2))
$4 = 0x4
(gdb) p/x (pte&(1<<3))
$5 = 0x8
```

关键发现： 由于 `R=1, W=1, X=1`，根据SV39规范，当RWX任一位为1时，该PTE是**叶子节点**，直接指向物理页而非下一级页表。因此这是一个**1GB大页映射**（在第一级就找到叶子项）。

步骤4：计算最终物理地址

```
ppn = pte >> PTE_PPN_SHIFT;
*physical = (ppn | (vpn & ((1L << ptshift) - 1))) << PGSHIFT;
(gdb) p/x ppn
$11 = 0x80000                            # 物理页号

(gdb) p/x *physical
$12 = 0x80200000                         # 最终物理地址
```

```
334          *physical = (ppn | (vpn & ((1L << ptshift) - 1))) << PGSHIFT
;
(gdb) s
337          if ((pte & PTE_R) || ((pte & PTE_X) && mxr)) {
(gdb) p/x ppn
$11 = 0x80000
(gdb) p/x *physical
$12 = 0x80200000
(gdb)
```

4.4 地址翻译结果验证

项目	值
虚拟地址	0xffffffffc02000d8
物理地址	0x80200000 + 偏移

项目	值
页表级数	1级（大页映射）
页面大小	1GB (2 ³⁰ bytes)

映射关系验证：

uCore内核使用的地址映射公式：

PA = VA - 0xFFFFFFFF40000000

验证：

0xffffffffc02000d8 - 0xFFFFFFFF40000000 = 0x802000d8 ✓

由于使用1GB大页，物理地址的计算为：

- PPN提供高位：0x80000 << 12 = 0x80000000
- 虚拟地址提供30位偏移：0xc02000d8 & 0x3FFFFFFF = 0x02000d8
- 最终：0x80000000 + 0x02000d8 = 0x802000d8

4.5 关键代码路径总结

```
get_physical_address() [target/riscv/cpu_helper.c:158]
├─ 检查分页模式 (vm = get_field(env->satp, SATP_MODE))
├─ 获取根页表基址 (base = get_field(env->satp, SATP_PPN) << PGSHIFT)
├─ for (i = 0; i < levels; i++) // sv39: levels=3
│   └─ 计算VPN索引: idx = (addr >> ptshift) & VPN_MASK
│       └─ 计算PTE地址: pte_addr = base + idx * 8
│           └─ 读取PTE: pte = ldq_phys(cs->as, pte_addr)
│               └─ 检查有效位: if (!(pte & PTE_V)) → Page Fault
│                   └─ 检查是否叶子节点: if (pte & (PTE_R | PTE_W | PTE_X))
│                       └─ 是叶子 → 跳出循环, 计算物理地址
│                           └─ 非叶子 → base = (pte >> PTE_PPN_SHIFT) << PGSHIFT
│                               继续下一级查找
```

4.6 实验总结

通过本次调试，我们观察到：

1. QEMU软件模拟了完整的SV39页表遍历过程，代码位于 target/riscv/cpu_helper.c
2. uCore使用1GB大页映射内核空间，仅需1次页表查询即可完成地址翻译，大大提高了效率

3. 地址翻译的核心步骤：

1. 从SATP寄存器获取根页表物理地址
2. 使用虚拟地址的VPN字段索引页表
3. 检查PTE的RWX位判断是否为叶子节点
4. 组合PPN和页内偏移得到物理地址

4. 调用链显示：本次触发来自GDB的内存读取请求，经过多层封装最终调用到

`get_physical_address`

五、实验关键问题与大模型辅助解决

问题1：带调试符号的QEMU编译失败

问题描述：在初次编译带调试信息的QEMU时，由于未清除之前编译的中间文件导致编译冲突。

大模型辅助：

- 建议执行 `make distclean` 清除所有中间文件，重新配置编译环境
- 确认configure参数： `--enable-debug` 确保包含完整调试信息
- 避免覆盖系统版本：提示不要执行 `sudo make install`，直接使用编译目录下的可执行文件

问题2：双GDB调试同步困难

问题描述：在启动两个GDB会话后，控制权切换不明确，经常出现一个会话阻塞另一个的情况。

大模型辅助：

- 明确流程顺序：
 - 终端1启动QEMU (`make debug`)
 - 终端2附加到QEMU进程 (直接continue并不设置断点)
 - 终端3连接ucore内核 (`make gdb`)，设置断点并执行到目标位置
 - 终端2设置QEMU内部断点后再继续执行
- 信号处理建议：在GDB2中执行 `handle SIGPIPE nostop noprint` 避免QEMU因管道信号频繁中断

问题3：实验现象与理论不符

问题描述：实验中观察到第一次页表查询就找到了叶子节点，与预期三级遍历不符。

大模型辅助解释：

- ucore内核的特殊映射：
 - uCore为内核空间设置了1GB大页直接映射
 - 目的是提高内核地址转换效率，减少TLB压力
 - 映射公式： $PA = VA - 0xFFFFFFFF40000000$

分支任务2：gdb 调试系统调用以及返回

一、实验目的

这次实验的主要目的是通过双重GDB调试的方式，深入观察QEMU模拟器是如何处理RISC-V架构中的系统调用机制的。具体来说，我需要：

1. 观察 `ecall` 指令是如何触发系统调用，实现从用户态（U-mode）到内核态（S-mode）的特权级切换
2. 观察 `sret` 指令是如何让系统从内核态返回用户态
3. 理解QEMU作为软件模拟器，是如何通过代码来模拟这些硬件行为的
4. 掌握双重GDB调试的技巧，一个GDB调试ucore内核，另一个GDB调试QEMU本身

说实话，一开始看到这个实验要求的时候我是有点懵的。不过按照指导书的步骤一步步来，最后还是成功观察到了整个系统调用的过程。

二、实验环境准备

这部分在分支任务1中已经成功实现，这里不再赘述。

三、双重GDB调试策略

这次实验的核心仍然还是要同时使用两个GDB：

- **GDB 1 (终端2)**：使用 `riscv64-unknown-elf-gdb`，连接到QEMU提供的调试接口，用来调试运行在模拟器中的ucore内核
- **GDB 2 (终端3)**：使用普通的 `gdb`，直接附加到QEMU进程本身，用来调试QEMU的源码

这种架构让我可以先在ucore中停在某个关键指令（比如 `ecall`）之前，然后在QEMU中设置断点观察它是如何处理这条指令的。

整个调试流程需要三个终端窗口配合：

- **终端1**：运行 `make debug` 启动QEMU
- **终端2**：运行 `make gdb` 调试ucore内核
- **终端3**：运行 `sudo gdb` 附加到QEMU进程

四、观察ecall指令的处理过程

4.1 启动调试环境

首先在**终端1**启动QEMU：

```
make debug
```

这时QEMU会启动并等待GDB连接。

然后在**终端3**启动调试ucore的GDB：

```
make gdb
```

4.2 加载用户程序符号表

这里遇到了第一个问题。我想在用户态的 `syscall` 函数打断点，结果GDB告诉我找不到这个文件：

```
(gdb) b user/libs/syscall.c:18
No source file named user/libs/syscall.c.
```

查了一下才知道，原来 `make gdb` 默认只加载了内核的调试符号，用户程序的符号表并没有加载进来。解决方法是手动加载用户程序的符号表：

```
add-symbol-file obj/__user_exit.out
y
```

加载完成后就可以在用户态代码打断点了。

4.3 找到ecall指令

我在 `syscall` 函数打了断点：

```
b user/libs/syscall.c:18
c
```

```
(gdb) b user/libs/syscall.c:18
Breakpoint 2 at 0x8000f8: file user/libs/syscall.c, line 19.
(gdb) c
Continuing.

Breakpoint 2, syscall (num=num@entry=30) at user/libs/syscall.c:19
19      asm volatile (
(gdb) display/7i $pc
1: x/7i $pc
=> 0x8000f8 <syscall+32>:      ld      a0,8(sp)
0x8000fa <syscall+34>:      ld      a1,40(sp)
0x8000fc <syscall+36>:      ld      a2,48(sp)
0x8000fe <syscall+38>:      ld      a3,56(sp)
0x800100 <syscall+40>:      ld      a4,64(sp)
0x800102 <syscall+42>:      ld      a5,72(sp)
0x800104 <syscall+44>:      ecall
```

程序会在第一次执行到 `syscall` 函数时停下来。但是 `syscall` 是个C函数，我真正关心的是里面的内联汇编中的 `ecall` 指令。所以需要切换到汇编级别的单步执行：

```
display/7i $pc
si
si
si
```

执行几次 `si` 后，我看到了 `ecall` 指令：

```
=> 0x800104 <syscall+44>: ecall
```

```
(gdb) i r $pc
pc                0x800104 0x800104 <syscall+44>
(gdb) x/i $pc
=> 0x800104 <syscall+44>:      ecall
(gdb) si
0xfffffffffc0200ef8 in __alltraps () at kern/trap/trapentry.S:123
```

这里又遇到了一个小问题。因为我之前在 `kern_entry` 也打了个断点，结果在执行 `si` 的时候报错：

```
Warning:
Cannot insert breakpoint 1: Cannot access memory at address 0xfffffffffc0200000
```

```
(gdb) si
Warning:
Cannot insert breakpoint 1: Cannot access memory at address 0xfffffffffc0200000
```

原因是程序现在在用户态，无法访问内核地址空间的断点。解决办法就是删掉那个断点：

```
delete 1
```

4.4 在QEMU中设置断点

现在 `ucore` 停在了 `ecall` 指令之前，接下来需要在终端2启动另一个GDB来调试QEMU。

首先找到QEMU的进程ID：

```
pgrep -f qemu-system-riscv64
```

得到的PID是60683，然后启动GDB并附加到这个进程：

```
sudo gdb
```

在GDB中：

```
attach 60683
handle SIGPIPE nostop noprint
```

这个 `handle SIGPIPE` 是很重要的，如果不设置的话，QEMU会因为SIGPIPE信号频繁中断，根本没法正常调试。

附加成功后，按 `Ctrl+C` 暂停QEMU的执行，然后设置断点：

```
b riscv_cpu_do_interrupt
c
```

这个 `riscv_cpu_do_interrupt` 就是QEMU中处理中断和异常的核心函数。

4.5 执行ecall并观察

回到终端3，执行 `ecall` 指令：

```
si
```

这时候终端2应该会停在断点处！果然，我看到了：

```
Thread 2 "qemu-system-ris" hit Breakpoint 1, riscv_cpu_do_interrupt
(cs=0x5a075d651650)
  at /home/lht/qemu-4.1.1/target/riscv/cpu_helper.c:507
507     RISCV_CPU *cpu = RISCVCPU(cs);
```

```
(gdb) b riscv_cpu_do_interrupt
Breakpoint 1 at 0x5a072f73eee6: file /home/lht/qemu-4.1.1/target/riscv/cpu_helper.c, line 507.
(gdb) c
Continuing.
[Switching to Thread 0x71d089fff6c0 (LWP 60683)]

Thread 2 "qemu-system-ris" hit Breakpoint 1, riscv_cpu_do_interrupt (cs=0x5a075d651650)
  at /home/lht/qemu-4.1.1/target/riscv/cpu_helper.c:507
507     RISCV_CPU *cpu = RISCVCPU(cs);
```

太好了！现在QEMU停在了处理ecall的代码中。我可以开始单步执行观察了。

4.6 深入观察ecall的处理流程

首先看一下调用栈：

```
bt
```

可以看到调用链：

```
#0  riscv_cpu_do_interrupt
#1  cpu_handle_exception
#2  cpu_exec
#3  tcg_cpu_exec
```

```
(gdb) bt
#0  riscv_cpu_do_interrupt (cs=0x5a075d651650)
    at /home/lht/qemu-4.1.1/target/riscv/cpu_helper.c:507
#1  0x00005a072f6aa76d in cpu_handle_exception (cpu=0x5a075d651650, ret=0x71d089ffe97c)
    at /home/lht/qemu-4.1.1/accel/tcg/cpu-exec.c:506
#2  0x00005a072f6aadf9 in cpu_exec (cpu=0x5a075d651650)
    at /home/lht/qemu-4.1.1/accel/tcg/cpu-exec.c:712
#3  0x00005a072f65cfef in tcg_cpu_exec (cpu=0x5a075d651650) at /home/lht/qemu-4.1.1/cpus.c:1435
#4  0x00005a072f65d8a8 in qemu_tcg_cpu_thread_fn (arg=0x5a075d651650)
    at /home/lht/qemu-4.1.1/cpus.c:1743
#5  0x00005a072fadd29e in qemu_thread_start (args=0x5a075d667ce0) at util/qemu-thread-posix.c:502
#6  0x000071d09109caa4 in start_thread (arg=<optimized out>) at ./nptl/pthread_create.c:447
#7  0x000071d091129c3c in clone3 () at ../sysdeps/unix/sysv/linux/x86_64/clone3.S:78
```

这说明ecall触发了一个异常，QEMU正在处理它。

然后查看关键信息：

```
n # 执行到env初始化完成
p env->priv
```

结果显示 `env->priv = 0`，说明当前是User mode。

继续查看异常信息：

```
p cs->exception_index # 输出：8（这是ecall from U-mode的异常号）
p/x env->sepc          # 输出：0x800020（用户态的PC）
p/x env->scause        # 输出：0x8000000000000005（旧的scause值）
```

```
(gdb) p env->priv
$3 = 0
(gdb) p cs->exception_index
$4 = 8
(gdb) p/x env->sepc
$5 = 0x800020
(gdb) p/x env->scause
$6 = 0x8000000000000005
```

然后我开始单步执行，观察关键的处理步骤：

```
n
n
n
```

```
(gdb) n
545         cause = ecall_cause_map[env->priv];
(gdb)
549     trace_riscv_trap(env->mhartid, async, cause, env->pc, tval, cause < 16 ?
(gdb)
550         (async ? riscv_intr_names : riscv_excp_names)[cause] : "(unknown)");
(gdb)
549     trace_riscv_trap(env->mhartid, async, cause, env->pc, tval, cause < 16 ?
(gdb)
552     if (env->priv <= PRV_S &&
(gdb)
553         cause < TARGET_LONG_BITS && ((deleg >> cause) & 1)) {
(gdb)
555         target_ulong s = env->mstatus;
(gdb)
556         s = set_field(s, MSTATUS_SPIE, env->priv_ver >= PRIV_VERSION_1_10_0 ?
(gdb) p env->priv
$11 = 0
```

在第545行看到了这个关键操作：

```
cause = ecall_cause_map[env->priv];
```

这是在根据当前特权级映射异常原因。继续往下：

```
(gdb) n
559         s = set_field(s, MSTATUS_SIE, 0);
(gdb)
560         env->mstatus = s;
(gdb)
561         env->scause = cause | ((target_ulong)async << (TARGET_LONG_BITS - 1));
(gdb)
562         env->sepc = env->pc;
(gdb)
563         env->sbadaddr = tval;
(gdb)
564         env->pc = (env->stvec >> 2 << 2) +
(gdb)
565             ((async && (env->stvec & 3) == 1) ? cause * 4 : 0);
(gdb)
564         env->pc = (env->stvec >> 2 << 2) +
(gdb)
566         riscv_cpu_set_mode(env, PRV_S);
```

// 第555-560行：保存和设置mstatus寄存器

```
target_ulong s = env->mstatus;
s = set_field(s, MSTATUS_SPIE, ...);
s = set_field(s, MSTATUS_SPP, env->priv);
s = set_field(s, MSTATUS_SIE, 0);
env->mstatus = s;
```

// 第561-563行：保存上下文

```
env->scause = cause | ((target_ulong)async << (TARGET_LONG_BITS - 1));
env->sepc = env->pc; // 保存返回地址
env->sbadaddr = tval;
```

// 第564-565行：设置新的PC

```
env->pc = (env->stvec >> 2 << 2) + ...;
```

// 第566行：切换特权级！

```
riscv_cpu_set_mode(env, PRV_S);
```

执行到第566行之后，我再次查看特权级：

```
p env->priv
```

```
(gdb) p env->priv
$14 = 1
(gdb) p/x env->stvec
$15 = 0xfffffffffc0200ef4
(gdb) p/x env->pc
$16 = 0xfffffffffc0200ef4
```

输出变成了1！特权级成功从0（User mode）切换到了1（Supervisor mode）！

同时查看PC的变化：

```
p/x env->stvec # 输出: 0xfffffffffc0200ef4
p/x env->pc     # 输出: 0xfffffffffc0200ef4
```

PC已经跳转到了内核的中断处理入口 `stvec`！

这就是ecall的完整处理流程：

1. 保存用户态的PC到 `sepc`
2. 设置异常原因到 `scause`
3. 修改 `mstatus` 寄存器保存当前状态
4. 特权级从User mode切换到Supervisor mode
5. PC跳转到 `stvec`（内核中断处理入口）

五、观察sret指令的处理过程

5.1 准备观察sret

观察完ecall之后，需要删除断点避免后续干扰：

```
delete 1
c
```

然后回到终端2，在系统调用返回的地方打断点：

```
b __trapret
c
```

```
(gdb) b __trapret
Breakpoint 3 at 0xfffffffffc0200f64: file kern/trap/trapentry.S, line 131.
(gdb) c
Continuing.
```

程序会在系统调用处理完成后停在 `__trapret` 函数。这时需要找到 `sret` 指令：

```
display/10i $pc
si
si
si
```

当看到 `sret` 指令时停止。

```
3: x/10i $pc
=> 0xfffffffffc0200fba <__trapret+86>: sret
    0xfffffffffc0200fbe <forkrets>: mv sp,a0
--Type <RET> for more, q to quit, c to continue without paging--
    0xfffffffffc0200fc0 <forkrets+2>: j 0xfffffffffc0200f64 <__trapret>
    0xfffffffffc0200fc2 <kernel_execve_ret>: addi a1,a1,-288
    0xfffffffffc0200fc6 <kernel_execve_ret+4>: ld s1,280(a0)
    0xfffffffffc0200fca <kernel_execve_ret+8>: sd s1,280(a1)
    0xfffffffffc0200fce <kernel_execve_ret+12>: ld s1,272(a0)
    0xfffffffffc0200fd2 <kernel_execve_ret+16>: sd s1,272(a1)
    0xfffffffffc0200fd6 <kernel_execve_ret+20>: ld s1,264(a0)
    0xfffffffffc0200fda <kernel_execve_ret+24>: sd s1,264(a1)
```

5.2 在QEMU中设置sret断点

切换到终端2，按 `Ctrl+C` 暂停QEMU：

```
b helper_sret
c
```

```
(gdb) b helper_sret
Breakpoint 2 at 0x5a072f73d4b5: file /home/lht/qemu-4.1.1/target/riscv/op_helper.c, line 76.
(gdb) c
Continuing.
[Switching to Thread 0x71d089fff6c0 (LWP 60683)]

Thread 2 "qemu-system-ris" hit Breakpoint 2, helper_sret (env=0x5a075d65a060,
    cpu_pc_deb=18446744072637910970) at /home/lht/qemu-4.1.1/target/riscv/op_helper.c:76
76         if (!(env->priv >= PRV_S)) {
```

然后回到终端3执行 `sret`：

```
si
```

终端3成功停在了 `helper_sret` 函数！

5.3 观察sret的处理流程

查看当前状态：

```
p env->priv      # 输出: 1 (Supervisor mode)
p/x env->sepc    # 输出: 0x800108 (要返回的用户态地址)
p/x env->pc      # 输出: 0xfffffffffc0200fba (当前内核态PC)
```

```
(gdb) p cpu->env.priv
No symbol "cpu" in current context.
(gdb) p env->priv
$17 = 1
(gdb) p/x env->sepc
$18 = 0x800108
(gdb) p/x env->pc
$19 = 0xffffffffc0200fba
```

然后单步执行：

```
n
n
n
```

看到关键代码：

```
(gdb) n
92      mstatus = set_field(mstatus,
(gdb)
96      mstatus = set_field(mstatus, MSTATUS_SPIE, 0);
(gdb)
97      mstatus = set_field(mstatus, MSTATUS_SPP, PRV_U);
(gdb)
98      riscv_cpu_set_mode(env, prev_priv);
(gdb) p env->priv
$23 = 1
(gdb) n
99      env->mstatus = mstatus;
(gdb) p env->priv
$24 = 0
(gdb) p/x env->pc
$25 = 0xffffffffc0200fba
(gdb) n
101     return retpc;
```

// 第80行：获取返回地址

```
target_ulong retpc = env->sepc;
```

// 第91行：从mstatus获取之前的特权级

```
target_ulong prev_priv = get_field(mstatus, MSTATUS_SPP);
```

```
// 第92-97行: 修改mstatus
mstatus = set_field(mstatus, ...);
mstatus = set_field(mstatus, MSTATUS_SPP, PRV_U);

// 第98行: 切换特权级!
riscv_cpu_set_mode(env, prev_priv);

// 第99行: 保存mstatus
env->mstatus = mstatus;

// 第101行: 返回用户态地址
return retpc;
```

在第98行执行之前, `env->priv = 1` (Supervisor mode) 。

执行第98行之后:

```
p env->priv # 输出: 0 (User mode)
```

特权级成功切换回用户态!

虽然此时 `env->pc` 还是内核地址, 但 `return retpc` 会让CPU跳转到 `0x800108` (用户态地址) 。

```
(gdb) p/x env->pc
$26 = 0xfffffffffc0200fba
(gdb) n
0x000071d08a000122 in code_gen_buffer ()
(gdb) p/x env->pc
No symbol "env" in current context.
```

六、TCG指令翻译机制的深入理解

6.1 什么是TCG

在这次实验中, 我一直在调试QEMU如何处理ecall和sret指令, 但实际上QEMU处理指令的方式并不是简单的"解释执行"。QEMU使用了一个叫做TCG (Tiny Code Generator) 的组件来实现**动态二进制翻译**。

TCG的工作原理是:

```
RISC-V指令 → TCG中间表示(IR) → 宿主机指令(x86-64)
```

也就是说, QEMU会把RISC-V指令翻译成我的Linux宿主机能直接执行的x86-64指令, 然后让真实的CPU去执行这些翻译后的代码。这比纯粹的解释执行要快得多。

6.2 TCG在本次实验中的体现

在我调试ecall和sret时，发现它们都是通过调用C函数来实现的：

- ecall → `riscv_cpu_do_interrupt`
- sret → `helper_sret`

这是因为**这些特权指令涉及的操作太复杂，无法直接翻译成x86-64指令。**

让我们对比一下：

普通指令（可以直接翻译）：

```
# RISC-V
add a0, a1, a2

# TCG可以翻译成x86-64
add %rdi, %rsi # （寄存器映射后）
```

特权指令（无法直接翻译）：

```
# RISC-V
ecall

# x86-64的系统调用是syscall，但它们的机制完全不同：
# - RISC-V: scause/sepc/stvec/mstatus
# - x86-64: MSR寄存器/STAR/LSTAR等
# 根本无法对应！
```

所以QEMU的做法是：**遇到特权指令时，TCG生成一段"调用helper函数"的x86-64代码**，然后由这个helper函数用C代码来模拟硬件行为。

6.3 与lab2页表调试实验的联系

为了验证TCG的存在，我按照以下步骤操作：

步骤1：在终端2（调试QEMU）中设置断点

```
# 按Ctrl+C暂停QEMU
(gdb) b helper_sret
Breakpoint 1 at 0x...
(gdb) c
Continuing.
```

步骤2：在终端3（调试ucore）中执行到sret

```
(gdb) b kern/trap/trapentry.S:133 # sret指令的位置
Breakpoint 2 at 0xffffffffc0200fba
(gdb) c
Continuing.
Breakpoint 2, __trapret () at kern/trap/trapentry.S:133
133          sret
(gdb) si # 单步执行sret
```

步骤3：终端2触发断点，查看调用栈

当执行 `si` 后，终端2（QEMU的GDB）触发了 `helper_sret` 断点：

```
(gdb) bt
#0  helper_sret (env=0x5cad2fc81060, cpu_pc_deb=18446744072637910970)
    at /home/lht/qemu-4.1.1/target/riscv/op_helper.c:76
#1  0x0000702dea000122 in code_gen_buffer ()
#2  0x00005cad26213c49 in cpu_tb_exec (cpu=0x5cad2fc78650, itb=0x702dea000040 <code_gen_buffer+19>)
    at /home/lht/qemu-4.1.1/accel/tcg/cpu-exec.c:173
#3  0x00005cad26214a8f in cpu_loop_exec_tb (cpu=0x5cad2fc78650,
    tb=0x702dea000040 <code_gen_buffer+19>, last_tb=0x702df1b7c988, tb_exit=0x702df1b7c980)
    at /home/lht/qemu-4.1.1/accel/tcg/cpu-exec.c:621
#4  0x00005cad26214db5 in cpu_exec (cpu=0x5cad2fc78650)
    at /home/lht/qemu-4.1.1/accel/tcg/cpu-exec.c:732
#5  0x00005cad261c6fef in tcg_cpu_exec (cpu=0x5cad2fc78650) at /home/lht/qemu-4.1.1/cpus.c:1435
#6  0x00005cad261c78a8 in qemu_tcg_cpu_thread_fn (arg=0x5cad2fc78650)
    at /home/lht/qemu-4.1.1/cpus.c:1743
#7  0x00005cad2664729e in qemu_thread_start (args=0x5cad2fc8ece0) at util/qemu-thread-posix.c:502
#8  0x0000702df249caa4 in start_thread (arg=<optimized out>) at ./nptl/pthread_create.c:447
#9  0x0000702df2529c3c in clone3 () at ../sysdeps/unix/sysv/linux/x86_64/clone3.S:78
(gdb) █
```

```
(gdb) bt
#0  helper_sret (env=0x5cad2fc81060, cpu_pc_deb=18446744072637910970)
    at /home/lht/qemu-4.1.1/target/riscv/op_helper.c:76
#1  0x0000702dea000122 in code_gen_buffer ()
#2  0x00005cad26213c49 in cpu_tb_exec (cpu=0x5cad2fc78650, itb=0x702dea000040
<code_gen_buffer+19>)
    at /home/lht/qemu-4.1.1/accel/tcg/cpu-exec.c:173
#3  0x00005cad26214a8f in cpu_loop_exec_tb (cpu=0x5cad2fc78650,
    tb=0x702dea000040 <code_gen_buffer+19>, last_tb=0x702df1b7c988,
    tb_exit=0x702df1b7c980)
    at /home/lht/qemu-4.1.1/accel/tcg/cpu-exec.c:621
#4  0x00005cad26214db5 in cpu_exec (cpu=0x5cad2fc78650)
    at /home/lht/qemu-4.1.1/accel/tcg/cpu-exec.c:732
#5  0x00005cad261c6fef in tcg_cpu_exec (cpu=0x5cad2fc78650) at /home/lht/qemu-
4.1.1/cpus.c:1435
#6  0x00005cad261c78a8 in qemu_tcg_cpu_thread_fn (arg=0x5cad2fc78650)
    at /home/lht/qemu-4.1.1/cpus.c:1743
#7  0x00005cad2664729e in qemu_thread_start (args=0x5cad2fc8ece0) at util/qemu-
thread-posix.c:502
#8  0x0000702df249caa4 in start_thread (arg=<optimized out>) at
./nptl/pthread_create.c:447
#9  0x0000702df2529c3c in clone3 () at
../sysdeps/unix/sysv/linux/x86_64/clone3.S:78
```

调用栈分析：TCG的确凿证据

这个调用栈完整地展示了TCG的工作流程，让我们从下往上分析：

底层（#9 → #5）：QEMU的进程和线程管理

- #9 `clone3()`：Linux系统调用，创建QEMU的线程
- #8 `start_thread()`：pthreads库，启动线程
- #7 `qemu_thread_start()`：QEMU的线程入口
- #6 `qemu_tcg_cpu_thread_fn()`：TCG专用的CPU执行线程

- #5 tcg_cpu_exec(): TCG的CPU执行函数

关键层 (#4 → #2) : TCG的翻译和执行

- #4 cpu_exec(): QEMU的主执行循环, 负责取指令、翻译、执行
- #3 cpu_loop_exec_tb(): 循环执行翻译块 (Translation Block, TB)
- #2 cpu_tb_exec(): 执行单个翻译块, 注意参数 itb=0x702dea000040 <code_gen_buffer+19>
- #1 0x0000702dea000122 in code_gen_buffer (): 这就是TCG生成的代码!

最上层 (#0) : Helper函数

- #0 helper_sret(): C函数, 模拟sret指令的硬件行为

code_gen_buffer: TCG生成代码的缓冲区

帧1和帧2中的 code_gen_buffer 是理解TCG的关键:

1. 它是什么:

1. 一块在QEMU运行时动态分配的内存区域
2. 用于存储TCG翻译后的宿主机机器码 (x86-64)
3. 大小通常为几MB到几十MB

2. 它包含什么:

1. 翻译后的RISC-V指令对应的x86-64机器码
2. 对于sret这种无法直接翻译的指令, 包含"调用helper_sret"的x86-64代码

3. 为什么GDB显示的符号信息有限:

1. 这块内存是运行时动态生成的, 不在QEMU的可执行文件(ELF)中
2. 没有对应的调试符号表
3. GDB只能显示基址 code_gen_buffer 和偏移量 (如 +19、+122)

4. 地址分析:

```
帧1: 0x0000702dea000122 in code_gen_buffer ()
帧2: itb=0x702dea000040 <code_gen_buffer+19>
```

- 基址: 0x702dea000000 (code_gen_buffer的起始位置)
- 帧2的偏移: +64字节 (0x40)
- 帧1的偏移: +290字节 (0x122)
- 这些偏移表示不同的翻译块或同一个块的不同位置

七、实验总结与收获

7.1 系统调用的完整流程

通过这次实验, 我完整地观察了从用户态发起系统调用到返回用户态的全过程:

ecall阶段 (进入内核):

- 用户程序执行 ecall 指令

- 触发异常，进入 `riscv_cpu_do_interrupt` 函数
- 保存上下文 (PC → `sepc`, 设置 `scause`)
- 特权级切换: User mode (0) → Supervisor mode (1)
- PC跳转: 用户态地址 → 内核中断入口 (`stvec`)

内核处理阶段:

- 内核根据系统调用号执行相应的服务
- 准备返回数据

sret阶段 (返回用户态) :

- 内核执行 `sret` 指令
- 进入 `helper_sret` 函数
- 从 `mstatus` 恢复之前的特权级
- 特权级切换: Supervisor mode (1) → User mode (0)
- PC恢复: 从 `sepc` 恢复到用户态返回地址

7.2 QEMU的模拟机制

这次实验让我真正理解了软件模拟硬件的方式。QEMU并不是直接解释执行RISC-V指令，而是：

1. **TCG翻译**: 将RISC-V指令翻译成宿主机 (x86-64) 的指令
2. **Helper函数**: 对于特殊指令 (如 `ecall`、`sret`)，调用预定义的helper函数来模拟硬件行为
3. **状态维护**: 通过 `CPURISCVState` 结构体维护CPU的所有状态 (寄存器、特权级等)

从调用栈可以看到：

```
helper_sret
  ← code_gen_buffer (TCG生成的代码！)
    ← cpu_tb_exec
      ← cpu_exec
```

`code_gen_buffer` 就是TCG生成的翻译后的代码。

7.3 遇到的问题和解决

虽然前面在处理过程中已经提到了我遇到的部分问题，但是为了更好的反思和总结，这里重新详细的整理一下。

1. 用户程序符号表未加载

问题描述: 一开始我尝试在用户态的 `syscall` 函数处打断点时，GDB提示我找不到这个源文件：

```
(gdb) b user/libs/syscall.c:18
No source file named user/libs/syscall.c.
<future shared library load? (y or [n])
```

我当时有点懵，明明这个文件在项目里是存在的啊。后来问了大模型才明白（后面也是才发现实验手册里面有写），原来 `make gdb` 启动调试时，默认只加载了内核的符号表（也就是 `obj/kernel`），用户程序虽然被编译链接到内核中了，但它们的调试符号并没有自动加载进GDB。这就导致GDB认识内核的函数和变量，但不认识用户程序的。

解决方法： 需要手动告诉GDB加载用户程序的符号表。对于我这次调试的 `exit` 程序，执行：

```
add-symbol-file obj/__user_exit.out
y # 确认加载
```

加载完成后，GDB会显示：

```
Reading symbols from obj/__user_exit.out...
```

这时再打断点就成功了：

```
b user/libs/syscall.c:18
Breakpoint 1 at 0x8000f8: file user/libs/syscall.c, line 19.
```

额外收获： 通过这个问题我了解到，在lab5中，用户程序是通过"Link-in-Kernel"的方式编译到内核镜像中的，而不是像真实Linux那样存储在文件系统里。所以如果要调试不同的用户程序，需要加载对应的 `.out` 文件。

2. 跨特权级断点冲突

问题描述： 这是我遇到的第二个比较棘手的问题。为了确保内核能正常启动，我在 `kern_entry` 处打了一个断点。但当程序进入用户态，我执行 `si` 单步到 `ecall` 指令时，突然报错：

```
Warning:
Cannot insert breakpoint 1: Cannot access memory at address 0xffffffffc0200000
Command aborted.
```

一开始我还以为是GDB出问题了，后来问了大模型才明白：`kern_entry` 的地址是 `0xffffffffc0200000`，这是一个内核虚拟地址。当程序运行在用户态时，用户态的页表根本无法访问内核地址空间，所以GDB也没法在那里插入断点了。

解决方法： 既然这个断点已经完成了它的使命（确保内核启动），那就直接删掉：

```
info break # 先查看断点编号
delete 1   # 删除编号为1的断点
```

删除后就可以正常单步执行了。

经验教训： 在调试操作系统这种会切换特权级的程序时，要特别注意断点的生命周期。最好的做法是：

- 调试内核态时，只在内核地址打断点
- 调试用户态时，只在用户地址打断点
- 及时清理不需要的断点，避免后续冲突

3. 变量未初始化错误

问题描述： 在QEMU的 `riscv_cpu_do_interrupt` 函数中，我停在了第507行：

```
507    RISCVCPU *cpu = RISCVCPU(cs);
```

我想查看当前的特权级，于是执行：

```
p env->priv
```

结果报错：

```
Cannot access memory at address 0x1f92fbf6bcf
```

这个地址看起来就很不对劲，像是一个野指针。

后面我也问了大模型，发现我理解错了，`env` 变量要到第508行才定义。。。

```
507    RISCVCPU *cpu = RISCVCPU(cs); // <- 当前在这一行
508    CPURISCVState *env = &cpu->env; // <- env还没初始化呢
```

我在第507行就尝试访问 `env`，这时候 `env` 这个变量还不存在，GDB读到的是栈上的随机值，当然会报错。

解决方法： 多执行一次 `n`，让程序走到第508行之后：

```
n # 执行到第508行
p env->priv # 这时就能正常访问了
```

或者更稳妥的做法是直接通过 `cpu` 变量访问：

```
p cpu->env.priv
```

因为 `cpu` 在第507行就已经初始化好了。

经验总结： 在单步调试时要注意当前停在哪一行，以及要访问的变量在哪一行定义。特别是在C语言中，变量是在定义的那一行才分配和初始化的，之前访问它们会读到垃圾数据。这个问题虽然简单，但在调试过程中很容易忽略，浪费了我不少时间。

7.4 实验心得

这次实验最大的收获是学会了“**透视**”系统的方法。通过双重GDB，我可以同时观察两个层面：

- 在ucore层面看到CPU执行的指令
- 在QEMU层面看到这些指令是如何被模拟的

这种调试方法在调试复杂系统时非常有用，可以帮助理解系统的运行机制。虽然设置起来有点复杂，需要三个终端窗口配合，但是一旦运行起来，就能看到很多平时看不到的细节。

另外，这次实验也让我意识到，现代计算机系统的每一个看似简单的操作（比如系统调用），背后都有非常复杂的机制在支撑。一条 `ecall` 指令，触发了异常处理、上下文保存、特权级切换、PC跳转等一系列操作。而这些操作在QEMU中又是通过数百行C代码来模拟的。

最后，虽然这次实验过程有些曲折，遇到了不少问题，但每次解决问题之后都有一种"原来是这样"的恍然大悟感。这种通过调试来学习系统底层机制的方法，确实比单纯看书或者听课要深刻得多。